

OpenMP Tutorial

Taura-Lab Spring Training 2026

前田 優希

田浦研究室 M1

2026.06.26

この Tutorial の目標

- OpenMP の概念を理解する
- OpenMP の基本的な使い方を習得する
- もう少し一般的に、並列プログラミングにおいて意識すべきことを学習する

目次

▶ OpenMP とは？

ループ並列化

正しく並列化するために

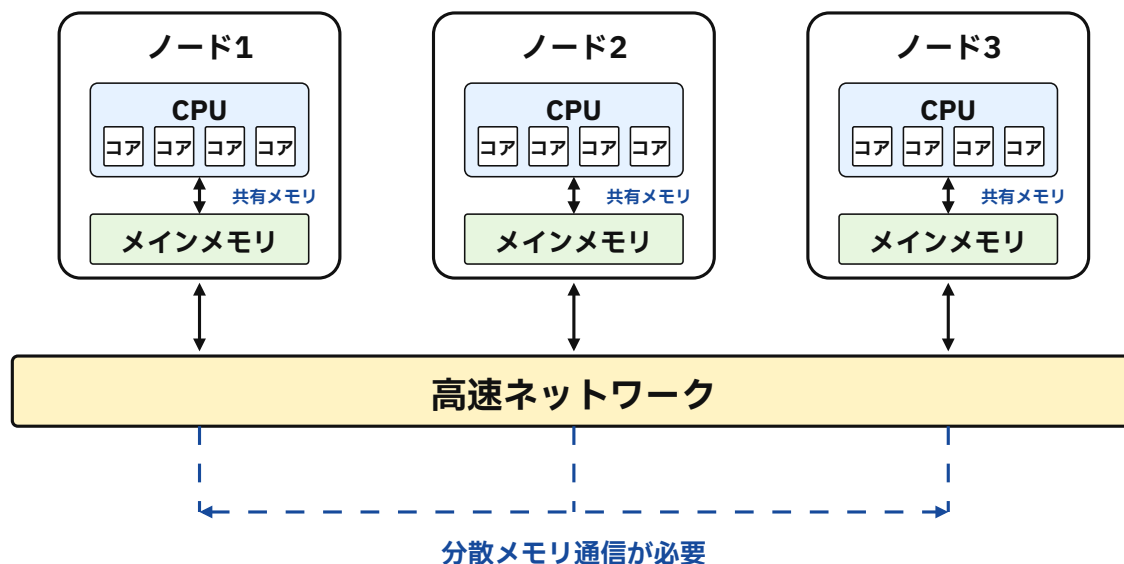
速く並列化するために

その他の構文

Appendix

HPC 計算機の基本構成

- HPC では、多数の計算機をネットワークで接続した計算クラスタを用いることが多い
- クラスタを構成する 1 台 1 台の計算機を**ノード**と呼ぶ
- 各ノードは、CPU・メモリ・場合によっては GPU などを持つ
 - CPU 内部には複数の**コア**があり、複数の処理を並列に実行できる
- 1 ノード内では複数コアがメモリを共有する (**共有メモリ**)
- 一方、ノード間ではメモリは共有されない (**分散メモリ**)



OpenMP とは？

- **OpenMP** とは、共有メモリ型並列計算機用にプログラムを並列化するための API 仕様・標準規格
 - MP は Multi-Processing を意味する
 - C, C++, Fortran から利用可能
- 長所：#pragma omp parallel などのコンパイラ指示文で並列処理を簡単に記述可能
 - **既存の逐次コードから少ない変更で，簡単に並列化を追加できる**
- 例：for 文の並列化

```
for (int i=0; i<N; i++) {  
    a[i] = i;  
}
```



```
#pragma omp parallel for  
for (int i=0; i<N; i++) {  
    a[i] = i;  
}
```

[補足] 「API 仕様・標準規格」とは？

- OpenMP は、特定の 1 つのライブラリ名ではなく、並列化のための**仕様**
- 仕様には以下が含まれる
 - `#pragma omp ...` のようなコンパイラ指示文
 - `omp_get_thread_num()` などのランタイム関数
 - `OMP_NUM_THREADS` などの環境変数
- 実際には、コンパイラやランタイムがこの仕様を実装する
 - GCC: `libgomp`
 - LLVM/Clang: `libomp`
 - Intel oneAPI: Intel OpenMP runtime
- OpenMP の規格書： <https://www.openmp.org/specifications/>

[補足] pragma とは

- pragma はコンパイラに対する指示文（**ディレクティブ**）
- C/C++では **#pragma** で始まる行として記述される
- OpenMP では **#pragma omp ...** の形式で並列化の指示をコンパイラに与える
- OpenMP が pragma を使うのは、既存の C/C++/Fortran プログラムに対して、元のコード構造を大きく変えずに並列化の指示を追加できるようにするため

ノード内並列化とノード間並列化

- OpenMP はノード内の並列化（**共有メモリ並列化**）を担う
- ノード間の並列化（**分散メモリ並列化**）を担当する API としては **MPI** などがある
- 実際の HPC では、ノード間は MPI、ノード内は OpenMP で並列化する**ハイブリッド並列**が重要
 - OpenMP 単体では主に 1 ノード内に制約され、大規模計算でのスケールアウトには MPI が必要
 - 一方で、分散メモリでは通信、データ配置、負荷分散を意識する必要があり、性能を出すのは容易ではない

#pragma omp parallel

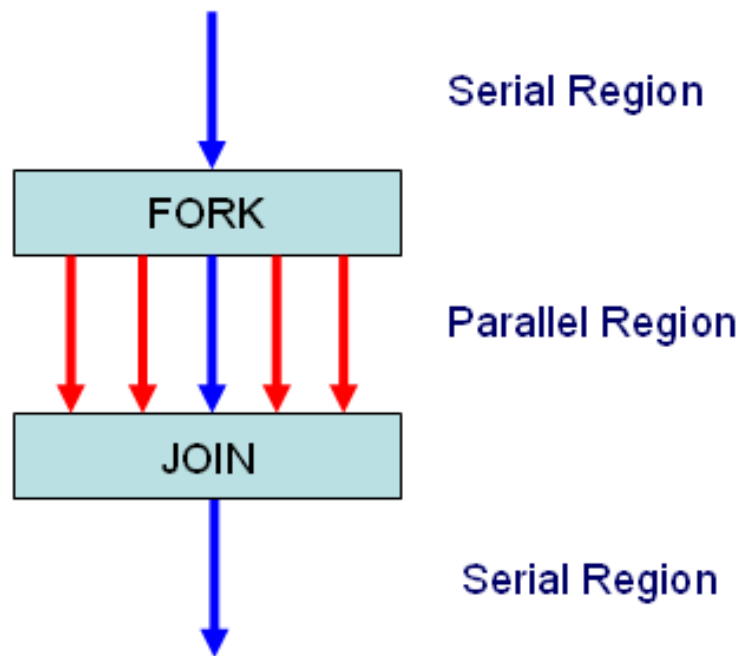
- 複数スレッドで実行する**並列領域**を作るための基本的な指示文
- 以下のように記述すると、並列領域内の文を各スレッドが実行する
- **同じ文がスレッドの数だけ実行される**点に注意
- ブロックの終わりで終了を待ち合わせる (**fork-join モデル**)

```
// sequential region

#pragma omp parallel
{
    // parallel region
}

// sequential region
```

Fork-Join モデル



- 並列領域に入ると，複数のスレッドに分岐 (**fork**)
- 各スレッドが並列に処理を実行
- 並列領域の終わりで同期 (**join**)

出典： <https://parallelcomputingsite.wordpress.com/2017/04/17/fork-join-in-open-mp/>

サンプルプログラム : hello.c

```
printf("Before parallel region\n");
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int nthreads = omp_get_num_threads();
    printf("Hello, World! I'm thread %d of %d\n", tid, nthreads);
}
printf("After parallel region\n");
```

- このように書くと、全スレッドが { } 内部の文を実行する
- `omp_get_num_threads()` で全体のスレッド数を取得できる
- `omp_get_thread_num()` で自分のスレッド番号を取得できる

コンパイルしてみよう

- OpenMP を使うには、コンパイル時に `-fopenmp` を付ける

```
$ gcc -fopenmp hello.c -o hello  
$ clang -fopenmp hello.c -o hello
```

- ただし、clang の場合は OpenMP ランタイムが別途必要なことがある
- 注意：macOS では話が面倒
 - macOS の gcc は、実は名前だけ gcc で中身は Apple Clang と呼ばれる clang になっている
 - この場合、上のようにはコンパイルできない
 - homebrew で gcc を入れて使うのが簡単

```
$ brew install gcc  
$ gcc-<your_gcc_version> -fopenmp hello.c -o hello
```

- サンプルプログラムでは、Makefile を `CC := gcc-<your_gcc_version>` に書き換えればよい

[補足] Apple Clang とは？

- macOS で以下を実行

```
$ gcc --version
Apple clang version 17.0.0 (clang-1700.6.3.2)
Target: arm64-apple-darwin25.1.0
Thread model: posix
InstalledDir: /Applications/Xcode.app/Contents/Developer/Toolchains/
XcodeDefault.xctoolchain/usr/bin
```

- Apple Clang は -fopenmp を直接サポートしていない
- 一応, Apple Clang で頑張ることもできる

```
$ brew install libomp
$ clang -Xpreprocessor -fopenmp -lomp \
-I$(brew --prefix libomp)/include \
-L$(brew --prefix libomp)/lib \
hello.c -o hello
```

実行してみよう

- 実行時は以下のように実行するスレッド数を指定できる

```
$ OMP_NUM_THREADS=4 ./hello
```

- あるいは、あらかじめ環境変数として設定しておくこともできる

```
$ export OMP_NUM_THREADS=4  
$ ./hello
```

- また、関数 `omp_set_num_threads(num_threads)` によりプログラム側でも指定できる

- 実行結果の例：

```
$ OMP_NUM_THREADS=4 ./hello
Before parallel region
Hello, World! I'm thread 1 of 4
Hello, World! I'm thread 2 of 4
Hello, World! I'm thread 0 of 4
Hello, World! I'm thread 3 of 4
After parallel region
```

- たしかに各スレッドが parallel 内部の文を実行している
- Before → Hello → After の順番は必ず守られる
 - これは fork-join モデルによるもの
 - Hello の中の順番は実行のたびに入れ替わる

実用的には？

- 実用上は、同じコード片を全スレッドで実行するだけでなく、異なる仕事をスレッド間で分担したい
- OpenMP では、`parallel` でスレッドチームを作り、その中で仕事を分割・分散する構文を使う
- 代表的な構文：
 - **for 構文**：`#pragma omp for`
ループのイテレーションを分割
 - **sections 構文**：`#pragma omp sections`
複数の独立処理ブロックを並列実行
 - **task 構文**：`#pragma omp task`
任意の処理を動的にタスク化して柔軟に分散
- なお、`for` や `sections` には、`parallel` とまとめた短縮形もある
 - `#pragma omp parallel for`
 - `#pragma omp parallel sections`
- `task` は通常、`parallel` 領域内で `single` から生成する

目次

OpenMP とは？

▶ **ループ並列化**

正しく並列化するために

速く並列化するために

その他の構文

Appendix

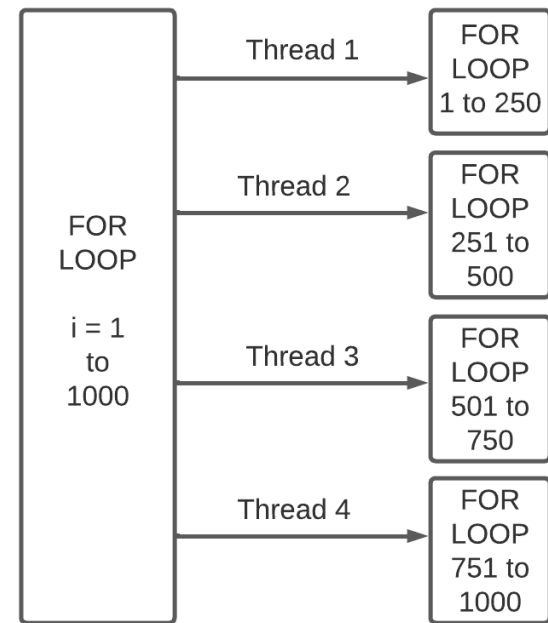
for 構文

- 以下のように書くことで, for 文の各イテレーションがスレッドに分散される

```
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0; i < N; i++) {
        a[i] = i;
    }
}
```

- 以下のように略記可能

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    a[i] = i;
}
```



出典 : <https://www.geeksforgeeks.org/c/c-parallel-for-loop-in-openmp/>

parallel_for.c を実行してみよう

- parallel_for.c を実行してみる
- 実行結果の例：

```
$ OMP_NUM_THREADS=4 ./parallel_for
Loop 2 is being processed by thread 1
Loop 3 is being processed by thread 1
Loop 4 is being processed by thread 2
Loop 5 is being processed by thread 2
Loop 0 is being processed by thread 0
Loop 1 is being processed by thread 0
Loop 6 is being processed by thread 3
Loop 7 is being processed by thread 3
```

- 若い番号から $\frac{\text{ループの個数}}{\text{スレッドの個数}}$ 個ごとにループが各スレッドに割り当てられる
- この割り当ての方法は変更可能（後述）

目次

OpenMP とは？

ループ並列化

▶ **正しく並列化するために**

▶ **共有変数とプライベート変数について**

そもそも並列化できない例

データ競合

速く並列化するために

その他の構文

Appendix

注意点①：共有変数とプライベート変数について

- OpenMP において、変数にはスレッドごとに共有されるもの（**共有変数**）とスレッドごとに独立に確保されるもの（**プライベート変数**）がある
 - これを変数の**属性**という
- `#pragma omp parallel for` の直後のループ変数以外はすべて共有変数となる(!)
- つまり、以下のようなプログラムは正しく動作しない
 - `j` が共有変数
 - `j = i` と `a[j] = i` の間に `j` が他スレッドにより書き換えられる可能性

```
int i, j;
#pragma omp parallel for
for (i = 0; i < N; i++) {
    j = i;
    a[j] = 1;
}
```

注意点①：共有変数とプライベート変数について

- 実際におかしなことが起こるのを確認する

```
#pragma omp parallel for
for (i = 0; i < 10000; i++) {
    j = i;
    a[j] = 1;
}

long sum = 0;
for (int idx = 0; idx < 10000; idx++) sum += a[idx];
```

- attribution_err.c の実行結果
 - 正しい結果は Sum = 10000

```
$ OMP_NUM_THREADS=4 ./attribution_err
Sum = 9986
$ OMP_NUM_THREADS=4 ./attribution_err
Sum = 9989
$ OMP_NUM_THREADS=1 ./attribution_err
Sum = 10000
```

注意点①：共有変数とプライベート変数について

- 正しくはこう， private 節で個別にプライベート変数に設定できる

```
int i, j;
#pragma omp parallel for private(j)
for (i = 0; i < N; i++) {
    j = i;
    a[j] = 1;
}
```

- あるいは default(private) とすればデフォルトがプライベート変数になる
- あるいは今回はこれでもいい
 - parallel 節内部で宣言された変数は private になるはず

```
int i;
#pragma omp parallel for
for (i = 0; i < N; i++) {
    int j = i;
    a[j] = 1;
}
```

[補足] 変数の属性

- **private** : 各スレッドに独立した領域を割り当て、**並列領域開始時の値は不定**
- **shared** : 全スレッドで同じ変数を参照、並列領域内で競合すると未定義動作になるため要同期
- **firstprivate** : private と同じだが、**並列領域開始時に親スレッドの値で初期化**
- **lastprivate** : 並列領域終了後、最後に更新されたスレッドの値を親スレッドに反映
- 詳細は `attribution.c` を参照

目次

OpenMP とは？

ループ並列化

▶ 正しく並列化するために

共有変数とプライベート変数について

▶ そもそも並列化できない例

データ競合

速く並列化するために

その他の構文

Appendix

注意点②：そもそも並列化できない例

- すべての for 文が並列化可能なわけではない
- i を 1 から N まで順番に回さないで逐次実行と結果が異なるようなプログラム（**流れ依存**）に対しては、（アルゴリズムの変更等を試みない限り）厳しい
- 例 1：

```
for (i = 1; i < N - 1; i++) {  
    a[i] = a[i] + 1;  
    b[i] = a[i - 1] + a[i + 1];  
}
```

- $a[i - 1]$ が更新されていない可能性がある

注意点②：そもそも並列化できない例

- 例 2 :

```
for (i = 0; i < N; i++) {  
    a[i] = a[b[i]];  
}
```

- `b[i]` の内容により並列化が可能かどうか決まる

目次

OpenMP とは？

ループ並列化

▶ 正しく並列化するために

共有変数とプライベート変数について

そもそも並列化できない例

▶ データ競合

速く並列化するために

その他の構文

Appendix

注意点③：データ競合

- 並列処理においては**データ競合**への対処が重要
- 例：Monte Carlo による π の推定
 - 各試行 i で `trial_hits(i)` (SUBSAMPLES 回ランダムで矢を投げる) を計算し、当たり数 `hits` を共有変数に足し合わせる
- 以下はデータ競合が発生するプログラムの例

```
long long hits = 0;

#pragma omp parallel for
for (int i = 0; i < N; i++) {
    long long h = trial_hits(i);
    hits += h; // data race!
}
```

- `hits` に対する読み込み→書き込みの間に他スレッドが書き込みを行う可能性がある
- OpenMP ではデータ競合を避ける方法がいくつかある

データ競合の回避① : lock

- OpenMP では lock 機構が用意されている
- 先ほどの例だと以下のように

```
omp_lock_t lock;  
omp_init_lock(&lock);  
#pragma omp parallel for  
for (int i = 0; i < N; i++) {  
    long long h = trial_hits(i);  
    omp_set_lock(&lock);  
    hits += h;  
    omp_unset_lock(&lock);  
}
```

データ競合の回避② : critical 補助指示文

- `#pragma omp critical` 内部の文は、同時には 1 つ以下のスレッドしか実行しない
- lock より自由度は低いが、実装は単純
- オプション (**name**) をつけることで複数のクリティカルセクションを作ることができる
 - 同じ name を持つクリティカルセクション同士は排他制御されるが、異なる name なら同時に実行できる
 - name は省略可能
- 先ほどの例だと以下のように

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    long long h = trial_hits(i);
    #pragma omp critical
    {
        hits += h;
    }
}
```

データ競合の回避③：atomic 補助指示文

- **単純な式一行のみ**（`x+=値` や `x^=値` など）であれば，`#pragma omp atomic` を用いると速い
- ハードウェア atomic 命令などで効率的に値を更新する
- 先ほどの例だと以下のよう

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    long long h = trial_hits(i);
    #pragma omp atomic
    hits += h;
}
```


比較してみる

- これらの実行時間を比較してみる
 - 実行時間の計測は `omp_get_wtime()` により可能
- `seq.c`, `lock.c`, `critical.c`, `atomic.c` を実行
 - `N = 20000000`, `SUBSAMPLES = 16`
- 実行結果の例：

```
$ ./seq
Elapsed time: 1.890952 seconds
$ ./lock
Elapsed time: 5.342480 seconds
$ ./critical
Elapsed time: 5.382293 seconds
$ ./atomic
Elapsed time: 1.273485 seconds
```

reduction 節

- 総和のように、複数スレッドの結果を最後に 1 つにまとめる場合は **reduction** 節を使うのが自然
- 先ほどの例では、hits に対する加算を以下のように書ける

```
long long hits = 0;

#pragma omp parallel for reduction(+:hits)
for (int i = 0; i < N; i++) {
    long long h = trial_hits(i);
    hits += h;
}
```

- reduction(+:hits) により、各スレッドは自分専用の hits を持つ
- 並列ループの終了時に、各スレッドの hits が足し合わされ、元の hits に反映される
- そのため、各反復で共有変数を直接更新する必要がない
 - atomic のように毎回共有変数を更新しないため、高速になりやすい

reduction のイメージ

- reduction は、概念的には以下のような処理に近い

```
long long hits_local[MAX_THREADS] = {0};

#pragma omp parallel
{
    int tid = omp_get_thread_num();

    #pragma omp for
    for (int i = 0; i < N; i++) {
        long long h = trial_hits(i);
        hits_local[tid] += h;
    }
}

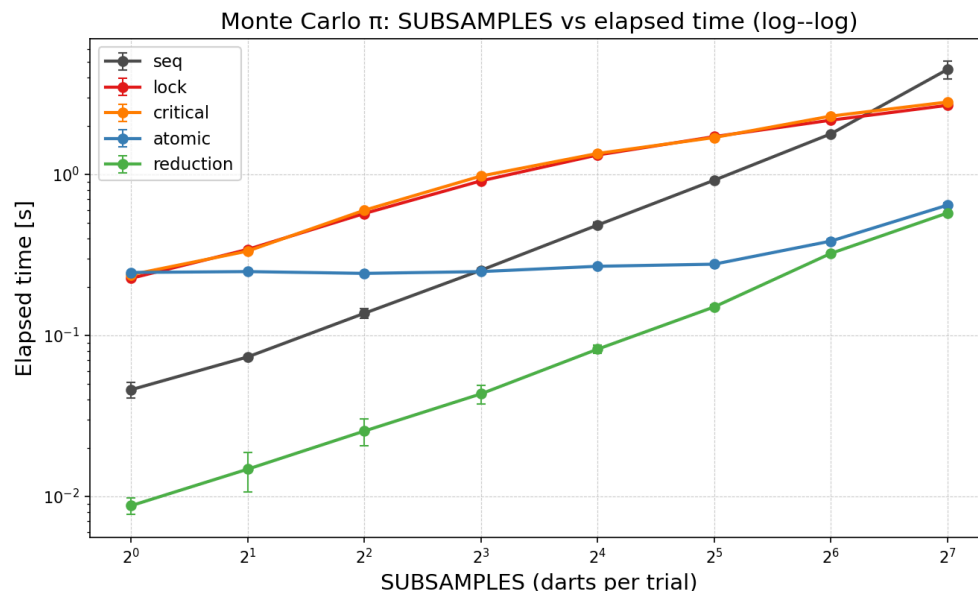
long long hits = 0;
for (int t = 0; t < nthreads; t++) {
    hits += hits_local[t];
}
```

- 同じ Monte Carlo 例で, `atomic.c` と `reduction.c` を比較
- 実行結果の例

```
$ ./atomic  
Elapsed time: 1.273485 seconds  
$ ./reduction  
Elapsed time: 0.340737 seconds
```

注意：いつもこうなるとは限らない

- 並列化の効果は、並列領域内の計算量に依存する
- SUBSAMPLES をいじることで trial_hits の計算量を変えることができる
 - 小さいと、各反復の計算が軽いため hits への同期コストが目立ち、並列版が seq より遅くなることもある
 - 大きいと、trial_hits(i) の計算が支配的になり、並列化の効果が出やすくなる



[補足] reduction として実行できる演算

- reduction では + 以外の演算も使える
- C/C++では, + * & | ^ && || max min が利用可能
- また, declare reduction により, ユーザが reduction 演算を定義できる

```
typedef struct {
    double x; double y;
} Pair;

#pragma omp declare reduction( \
    pair_plus : Pair : \
    omp_out.x += omp_in.x, omp_out.y += omp_in.y \
) initializer(omp_priv = {0.0, 0.0})

Pair total = {0.0, 0.0};
#pragma omp parallel for reduction(pair_plus:total)
for (int i = 0; i < N; i++) {
    Pair p = compute_pair(i);
    total.x += p.x;
    total.y += p.y;
}
```

目次

OpenMP とは？

ループ並列化

正しく並列化するために

・ **速く並列化するために**

・ **スケジューリング**

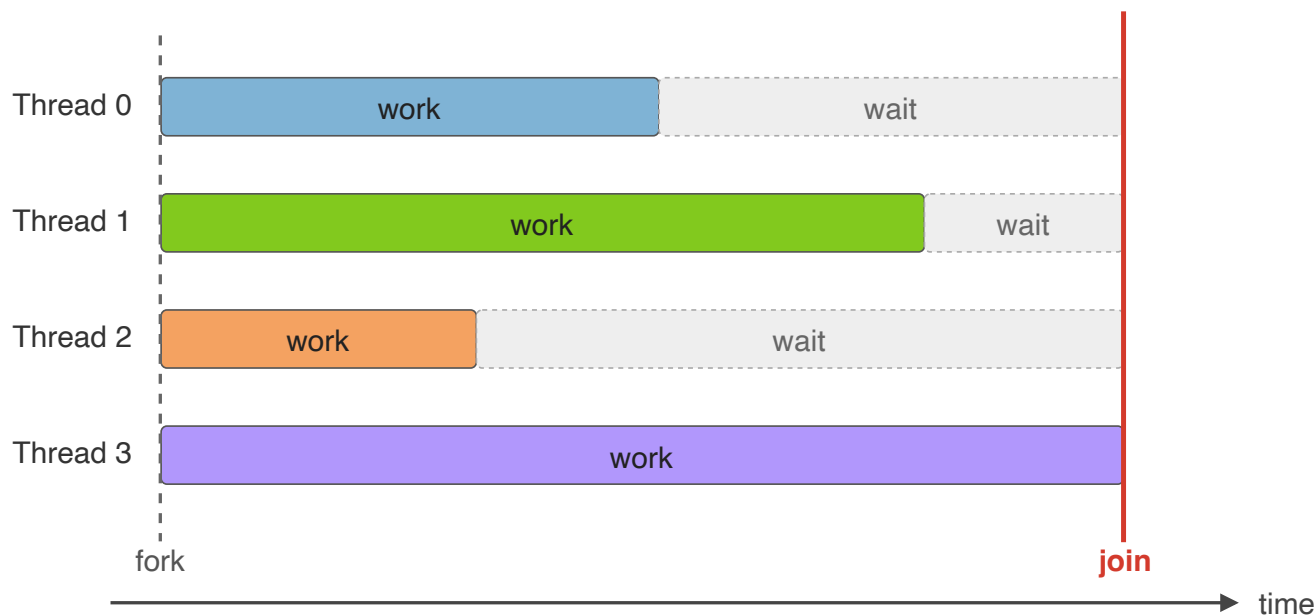
スレッド数のチューニング

その他の構文

Appendix

スケジューリングの重要性

- 高性能化のためにはスケジューリングが重要
- 並列化をしたとしても、負荷分散が不均衡だと性能が出ない
- 特に OpenMP の fork-join モデルでは、仕事の最も多いスレッドが終わるまで待たないといけない

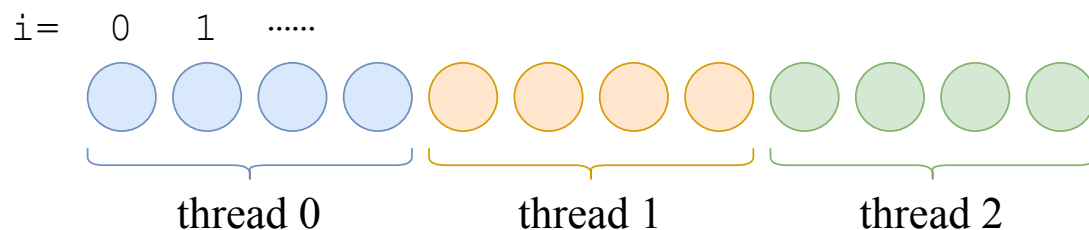


スケジューリングの種類

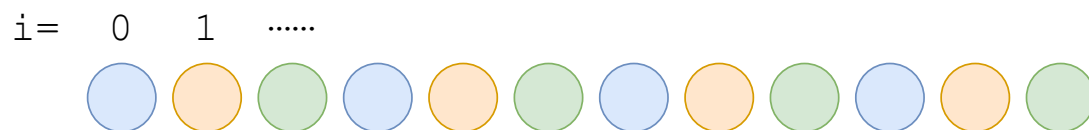
- for ループのスケジューリングの種類には以下の 3 通りがある
 1. **static** : スレッドごとに chunk 個のイテレーションを静的に割り当てる
 2. **dynamic** : スレッドごとに chunk 個のイテレーションを動的に割り当てる, 終わったスレッドから次の chunk を取りに行く
 3. **guided** : スレッドごとに割り当てるイテレーションの個数を徐々に小さくする
- `#pragma omp parallel for schedule(dynamic, CHUNK_SIZE)` のように指定できる
 - 省略した場合のデフォルトは, 仕様上は規定されていないが, 主要実装では `static` になる

- `schedule(static, CHUNK_SIZE)`

- ループを `CHUNK_SIZE` 個ごとに分割し、スレッド 0 番からラウンドロビンで**静的に**割り当てる
- `schedule` 補助指定文を省略したときのデフォルトは `static` で、`CHUNK_SIZE` は $\frac{\text{ループの個数}}{\text{スレッドの個数}}$ となる（要するに、何もスケジューリング句を指定しないようになる）
- $\text{CHUNK_SIZE} = \frac{\text{ループの個数}}{\text{スレッドの個数}}$ のとき

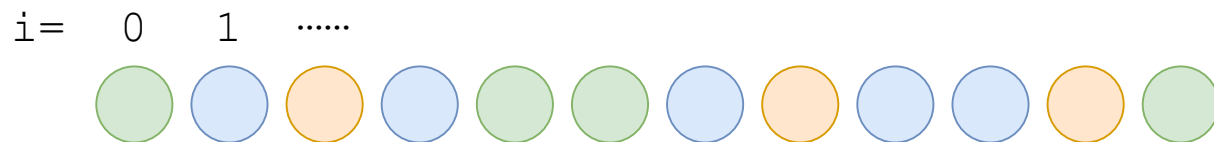


- $\text{CHUNK_SIZE} = 1$ のとき



- `schedule(dynamic, CHUNK_SIZE)`

- ループを `CHUNK_SIZE` 個ごとに分割し、**処理が終わったスレッドに順々に次の処理を割り当てる**
- 実行時にならないとループ内部の計算量が明らかにならない場合に有効
- `CHUNK_SIZE` を省略すると 1 になる



- `schedule(guided, CHUNK_SIZE)`
 - 徐々にチャンクサイズを小さくしながら、処理が終わったスレッドに順々に次の処理を割り当てる
 - `CHUNK_SIZE` はチャンクサイズの最小単位
 - チャンクサイズに 1 より大きい k を指定した場合、チャンクサイズは指数的に k まで小さくなるが、最後のチャンクは k より小さくなる場合がある
 - `CHUNK_SIZE` を省略すると 1 になる

i= 0 1



各スケジューリングの特徴

- dynamic, guided のチャンクサイズは性能に影響する
 - チャンクサイズ小→負荷分散は良くなるが、オーバーヘッドは大きくなる
 - チャンクサイズ大→負荷分散は悪くなるが、オーバーヘッドは小さくなる
- 割り当てのオーバーヘッド
 - dynamic, guided は実行時スケジューリングのため、static よりもオーバーヘッドが大きい
 - **事前に負荷分散が均衡となるスケジューリングを組むことができれば**, static スケジューリングを用いることでさらに高速化できる可能性がある

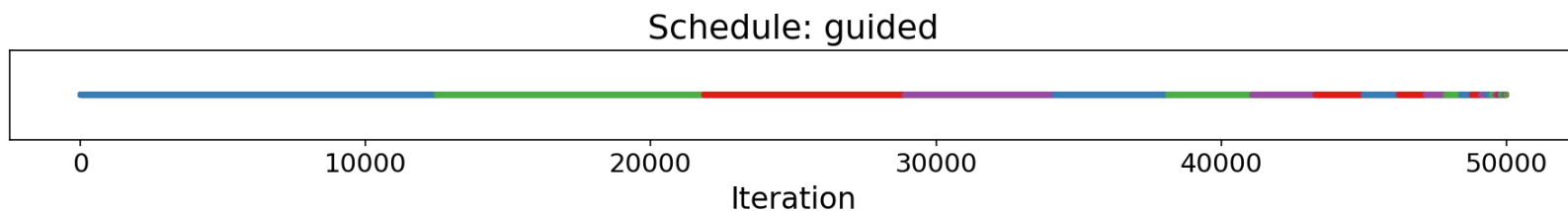
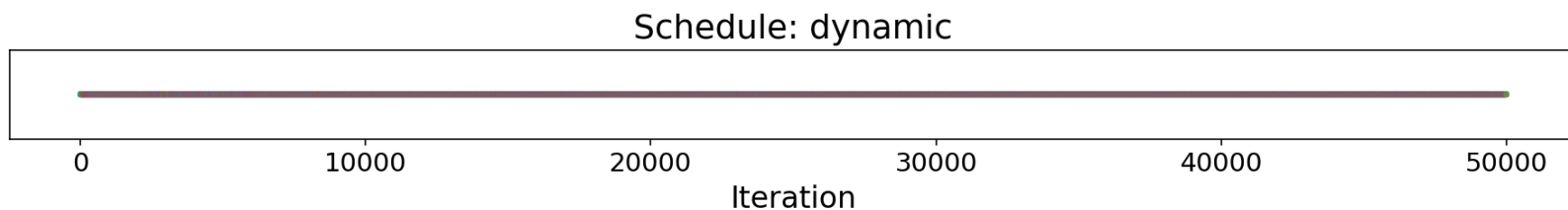
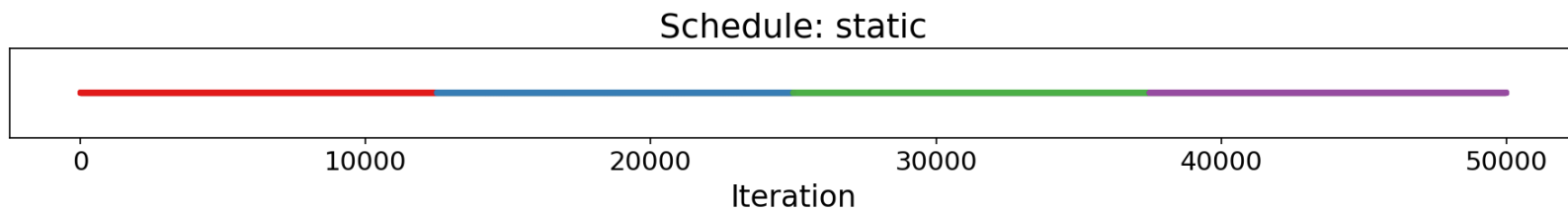
スケジューリングの比較

- 以下のような for ループの外側のみを OpenMP で並列化する
 - ループの後半ほど計算量が大きい
- スケジューリングとして上記の 3 パターンを試し，実行時間を比較する
- どのループをどのスレッドが担当したかの記録も取っておく

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < i; j++) {  
        A[i]++;  
    }  
}
```

スケジューリングの比較

- static.c, dynamic.c, guided.c を実行 (スレッド数は 4)



スケジューリングの比較：計測結果

- 実行結果の例：

```
=== OpenMP threads: 4 ===  
static : 0.372155 ± 0.004169 s  
dynamic: 0.586862 ± 0.003830 s  
guided : 0.259611 ± 0.067941 s
```

- 負荷分散は良好なのにもかかわらず、dynamic が static よりも遅い
 - スケジューリングのオーバーヘッドが大きいと考えられる
- guided が最速
 - 小さいオーバーヘッドと良好な負荷分散を両立している

static のチャンクサイズを変える

- static の `CHUNK_SIZE` を 1 とすればより均等に分割される（もともとのチャンクサイズは $\frac{\text{ループの個数}}{\text{スレッドの個数}}$ であったことに注意）
- 実行してみる

```
=== OpenMP threads: 4 ===  
static : 0.264197 ± 0.042221 s  
dynamic: 0.586862 ± 0.003830 s  
guided  : 0.259611 ± 0.067941 s
```

- 静的割り当てなぶん static は guided より速くなるかと思ったが、明確に速くなるわけでもなかった
 - キャッシュヒット率の違い？
guided は序盤は連続アクセス, static は 4 個飛びでバラバラ

参考：collapse 補助指示文

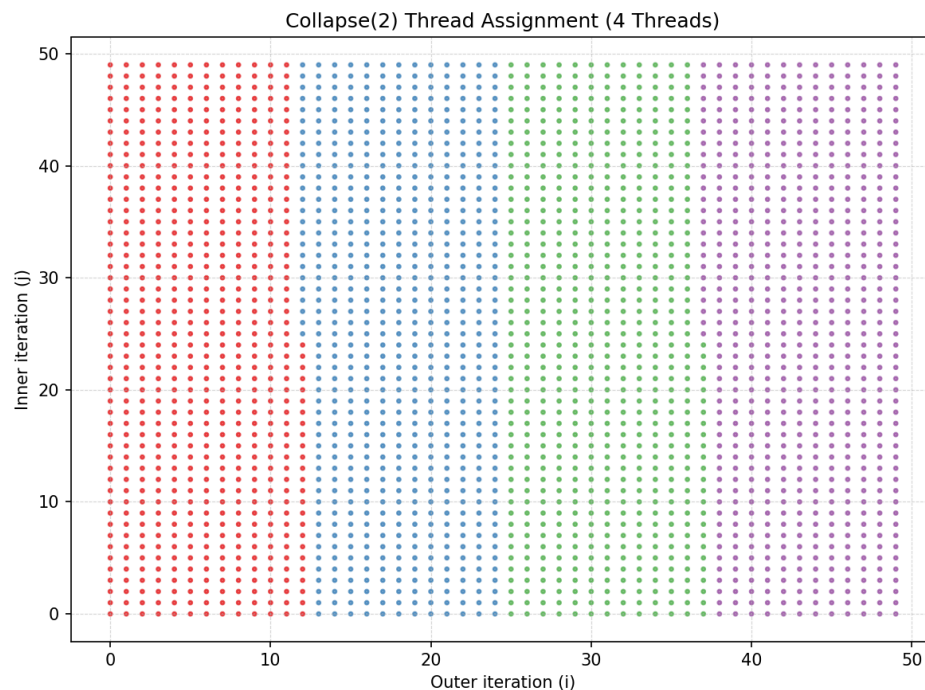
- ネストされたループを並列化する際は collapse 補助指示文を用いることができる

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        C[i][j] = A[i][j] + B[i][j];
    }
}
```

- for 文の間に何か操作を挟むことはできない（純粋な二重ループでないといけない）
- collapse(n) で n 層のループを**大きな一つのループとみなして**並列化できる
- schedule 補助指示文と組み合わせることもできる

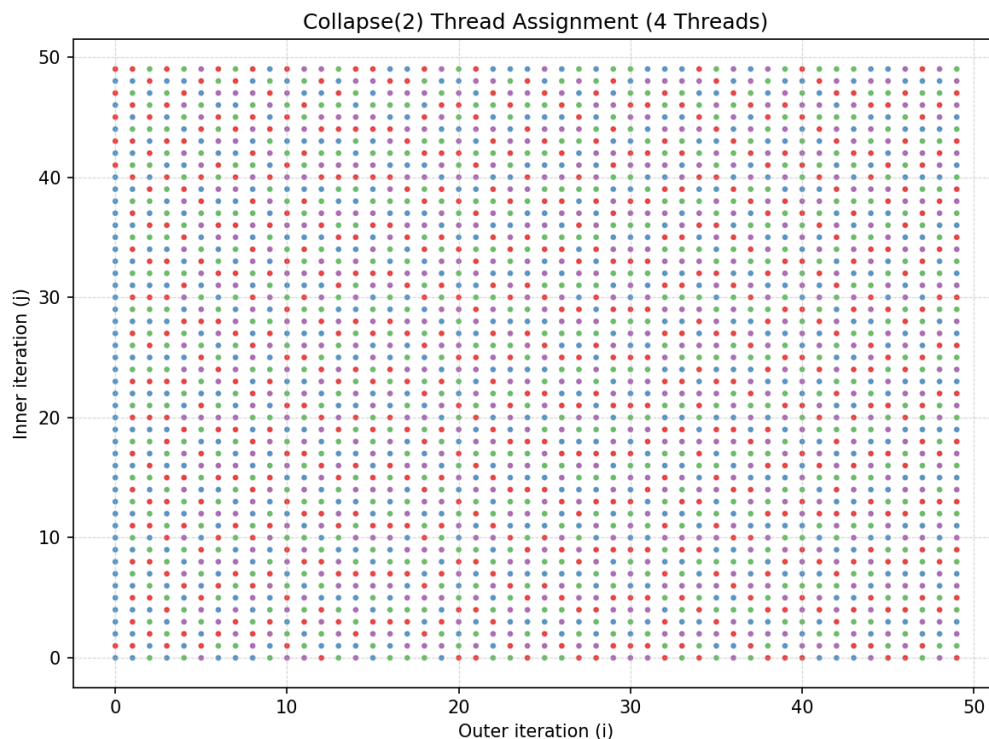
collapse の挙動を調べてみる

- collapse.c を実行
- スレッドごとに色分けしてどのループを担当しているか可視化
- schedule の指定なし（自動的に static になる）での実行結果：



collapse の挙動を調べてみる

- `schedule(dynamic, 1)` での実行結果：



- 二重ループ的な構造はもはや関係ない

- 二重ループを展開することで、分散すべきイテレーションの個数が増える
- 上記のような CPU 上のプログラムではあまり嬉しさはないかもしれないが、役に立つ例としては **GPU 上での for 文の並列化**などが考えられる
- 詳細は Appendix の例を参照

目次

OpenMP とは？

ループ並列化

正しく並列化するために

・ **速く並列化するために**

スケジューリング

・ **スレッド数のチューニング**

その他の構文

Appendix

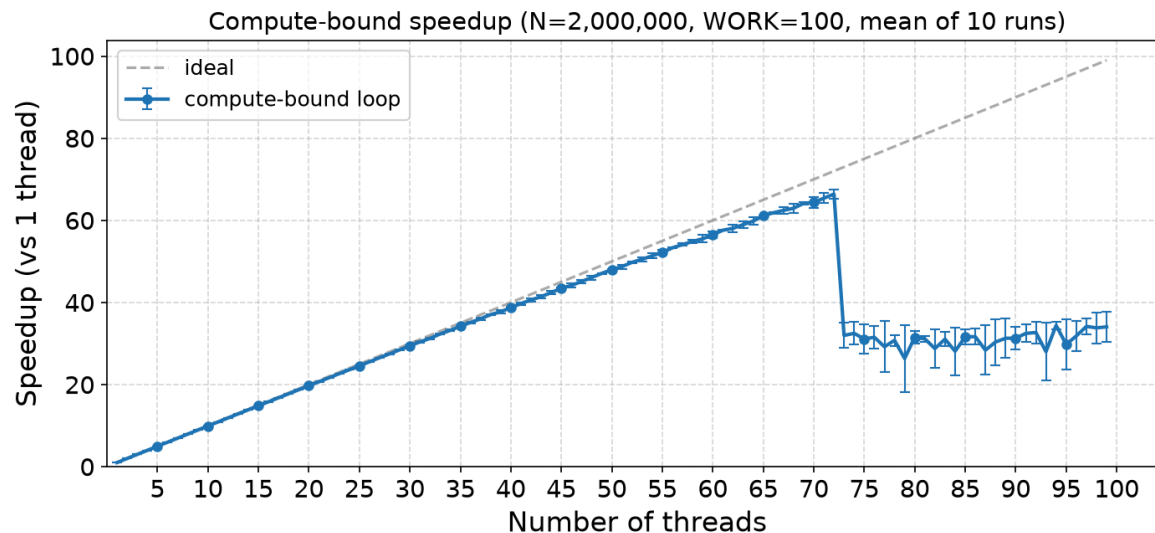
最適なスレッド数は？

- 話は変わって、最適なスレッド数について考えてみる
- スレッドを増やすほど性能が向上するわけではない
 - そもそも**論理コアの数**までしかスレッドは同時に実行できない
- 利用可能な論理コア数を知るためのコマンド
 - Linux : `nproc`
 - Mac OS : `sysctl -n hw.logicalcpu`
- Miyabi-g では論理コア数は 72

スレッド数と実行時間の関係①：計算が重い例

- 各要素で重い計算を行う `measure_threads_compute.c` をスレッド数を変えながら実行
 - 十分に計算量が大きいため、並列化の効果が大きいことが期待される

```
WORK = 100
#pragma omp parallel for
for (long i = 0; i < N; i++) {
    double x = 1.0 + 1.0e-9 * i;
    for (int k = 0; k < WORK; k++) {
        x = x * 1.0000001 + 0.0000001;
        x = x * 0.9999999 + 0.0000002;
    }
    out[i] = x;
}
```

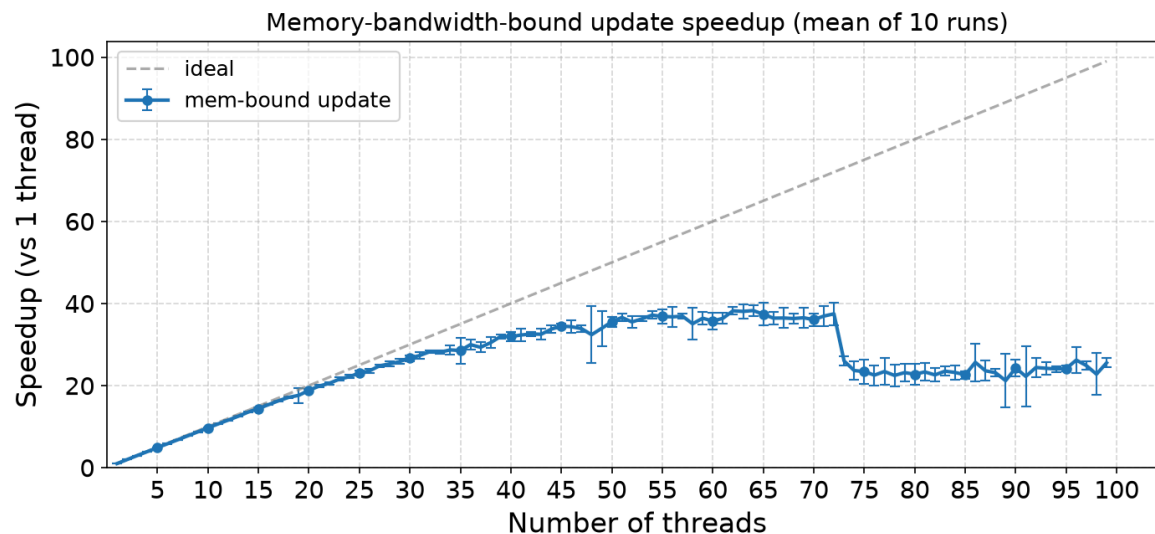


- スレッド数を増やすと speedup も増えるが、論理コア数付近で ideal speedup から離れる

スレッド数と実行時間の関係②：メモリ帯域の影響

- スレッド数を増やしても、必ず性能が伸び続けるわけではない
- 例：大きな配列に対する単純な演算 (measure_threads_mem.c)
 - この処理は計算量に比べてメモリアクセスが多い
 - $B[i], C[i]$ を読み, $A[i]$ に書く
- スレッドを増やすと, CPU コアではなく **メモリ帯域** が先に限界になることがある

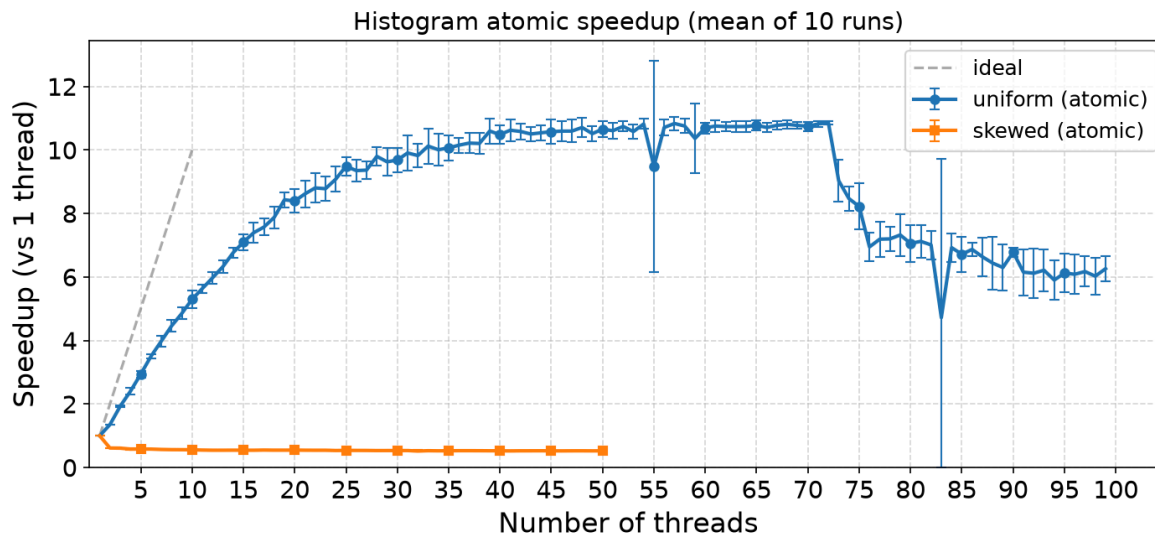
```
#pragma omp parallel for
for (long i = 0; i < N; i++) {
    A[i] = B[i] + 2.0 * C[i];
}
```



スレッド数と実行時間の関係③：競合の影響

- 正しく同期していても、スレッド数を増やすほど速くなるとは限らない
- 例：ヒストグラム計算 (measure_threads_hist.c)
- atomic によりデータ競合は防げる
- しかし、複数スレッドが同じ hist[bin] を頻繁に更新すると、その場所がボトルネックになる
- データが一部の bin に偏っていると、スレッド数を増やすほど競合が激しくなる

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    int bin = data[i];
    #pragma omp atomic
    hist[bin]++;
}
```



目次

OpenMP とは？

ループ並列化

正しく並列化するために

速く並列化するために

▶ **その他の構文**

Appendix

- 並列領域の中で、1 スレッドにだけ実行させたい処理を書く際に使う
- single 構文の内部は、1 つのスレッドのみが実行する
- どのスレッドが実行するかは不定
- nowait 補助指定文を用いない限り、**single 終了後に同期が入る**
- 詳細は single.c も参照

```
#pragma omp parallel
{
    #pragma omp single
    {
        work();
    }
}
```

masked 構文

- single 構文と似ており，チーム内の一部のスレッドだけが実行する
- 節を指定しない場合，**thread 0 が実行する**
 - filter 節により，実行するスレッドを指定できる
- **終了後に暗黙の同期は入らない**
- 詳細は `masked.c` も参照
 - なお，以前使われていた `master` 構文は deprecated であり，OpenMP 5.1 からは `masked` の使用が推奨される

```
#pragma omp parallel
{
    #pragma omp masked
    {
        work();
    }
}
```

ループ以外の並列化①：section 構文

- for 構文よりもう少し自由度の高い(?)並列化方法
- 各 section がスレッドに割り当てられる

```
#pragma omp parallel sections
{
    #pragma omp section
    {
        work1();
    }
    #pragma omp section
    {
        work2();
    }
}
```

- 詳細は略

ループ以外の並列化②：task 構文

- いわゆる「タスク並列」のための構文
- これまでは、主に for ループの各反復をスレッドに分配する方法を見てきた
- しかし、並列化したい処理がいつも for ループの形で書けるとは限らない
 - 再帰的な分割統治法
 - 木構造・グラフ構造の探索
 - 実行中に新しい仕事が動的に生まれる処理
- task 構文を使えば、並列化の単位をより柔軟に設定できる

ループ以外の並列化②：task 構文

- 例：フィボナッチ数列の計算

```
int fib(int n) {
    if (n < 2) return n;
    else{
        #pragma omp task shared(x) firstprivate(n)
        x = fib(n - 1);
        #pragma omp task shared(y) firstprivate(n)
        y = fib(n - 2);
        #pragma omp taskwait
        return x + y;
    }
}

#pragma omp parallel
{
    #pragma omp single
    printf("Fibonacci of %d is %d\n", 10, fib(10));
}
```

- task** は、「あとで誰かが実行できる仕事」を作るための構文
 - 注：task は、新しいスレッドを作る指示ではない
 - 生成されたタスクは、OpenMP ランタイムが既存のスレッドに割り当てて実行する
- taskwait** により、生成した子タスクの完了を待つことができる

ループ以外の並列化②：task 構文

- 例：フィボナッチ数列の計算

```
int fib(int n) {
    if (n < 2) return n;
    else{
        #pragma omp task shared(x) firstprivate(n)
        x = fib(n - 1);
        #pragma omp task shared(y) firstprivate(n)
        y = fib(n - 2);
        #pragma omp taskwait
        return x + y;
    }
}

#pragma omp parallel
{
    #pragma omp single
    printf("Fibonacci of %d is %d\n", 10, fib(10));
}
```

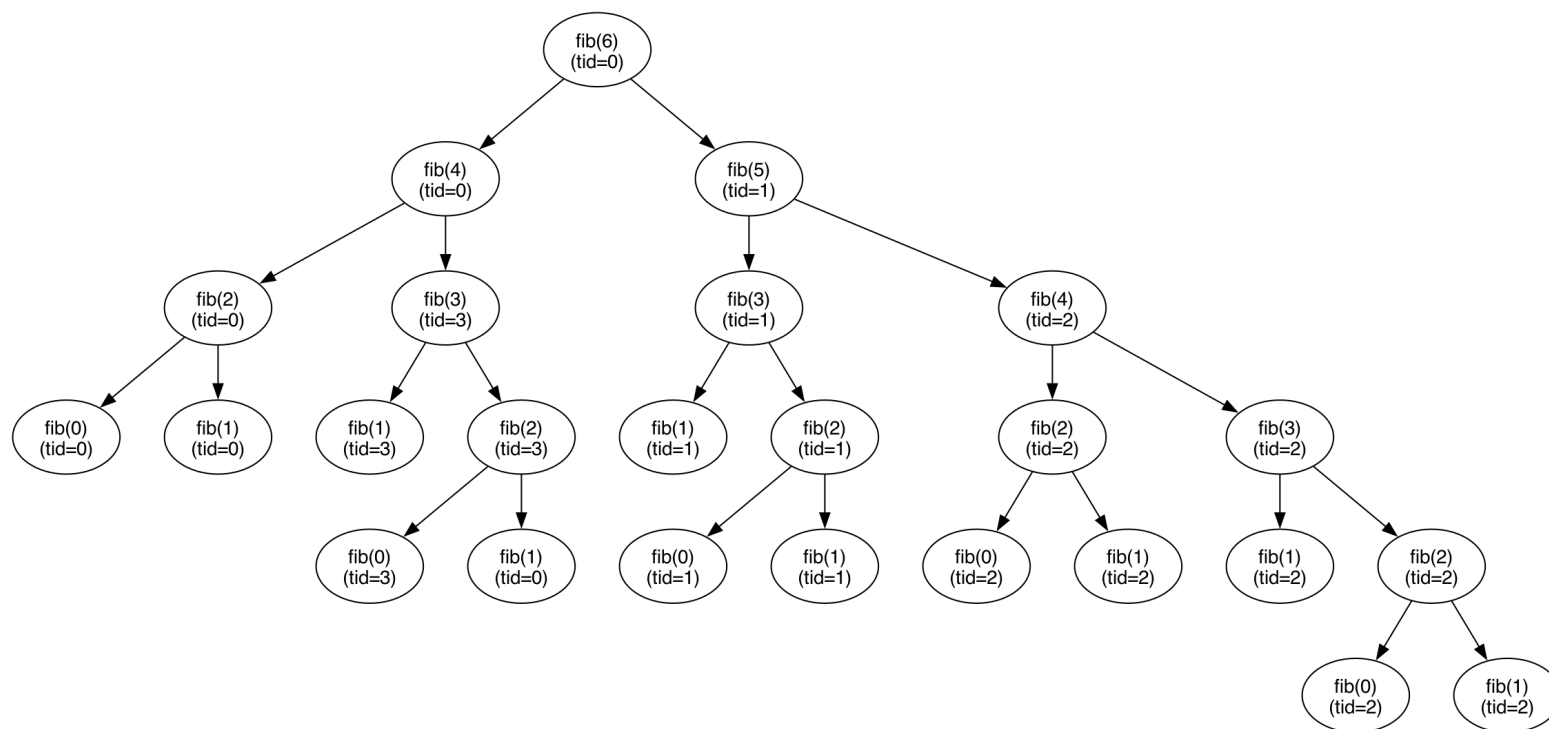
- 全スレッドが fib(10) を呼ぶと、同じ計算を重複して実行してしまう
- そのため、single または masked により、最初のタスク生成は 1 スレッドだけが行う
- 生成されたタスクは、他の待機中のスレッドも実行できる

- taskwait は、現在のタスクが生成した子タスクの完了を待つ
- Fibonacci の例では、fib(n - 1) と fib(n - 2) の結果が揃うまで待つ必要がある
- taskwait がないと、x や y が計算される前に x + y を返してしまう可能性がある

```
#pragma omp task shared(x) firstprivate(n)
x = fib(n - 1);
#pragma omp task shared(y) firstprivate(n)
y = fib(n - 2);
#pragma omp taskwait
return x + y;
```

タスク実行の可視化

- fib_tree.c を実行し, animate_fib.py で タスク生成の様子を可視化してみる



- GIF へのリンク

[補足] Fibonacci 例の注意点

- Fibonacci は、task 構文の動きを説明するには分かりやすい
- しかし、実用的な高性能化の例としてはあまり良くない
 - そもそもタスク並列で計算する必然性がない
 - タスクが細かすぎるため、計算量に対してタスク生成オーバーヘッドが支配的
 - 現実のタスク並列プログラムらしくない
- むしろ、ランタイムのオーバーヘッドを測る目的でよく用いられる
- より「ちゃんとした」タスク並列のベンチマークとしては BOTS などを参照

task 生成の粒度制御

- タスクの作成や管理にはオーバーヘッドが生じる
 - タスクの問題サイズが十分小さければ、いちいちタスク化して分配するより、逐次実行した方が速い
- そのため、実用上はこのような閾値（**cutoff**）を設けることが多い

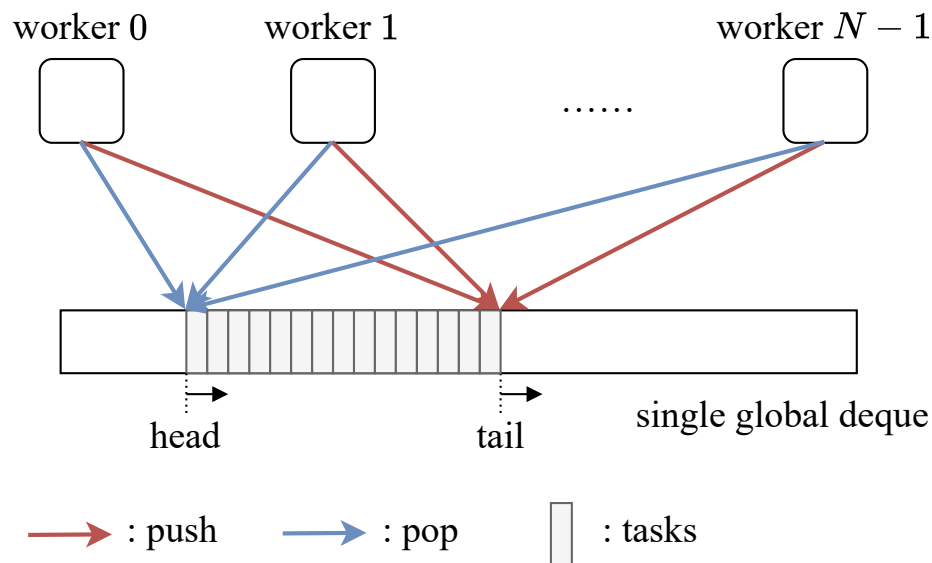
```
int fib(int n) {  
    if (n < 20) {  
        return fib_serial(n);  
    }  
  
    int x, y;  
    #pragma omp task shared(x) firstprivate(n)  
    x = fib(n - 1);  
    #pragma omp task shared(y) firstprivate(n)  
    y = fib(n - 2);  
    #pragma omp taskwait  
    return x + y;  
}
```

- OpenMP の task には if 節を付けることもできる
- 条件が真なら通常のタスクとして生成される
- 条件が偽なら、基本的にはその場で実行されるタスクになる
- 手動の cutoff と同様に、細かすぎるタスクの生成を避ける目的で使える

```
#pragma omp task if(n > 20) shared(x) firstprivate(n)
x = fib(n - 1);
#pragma omp task if(n > 20) shared(y) firstprivate(n)
y = fib(n - 2);
```

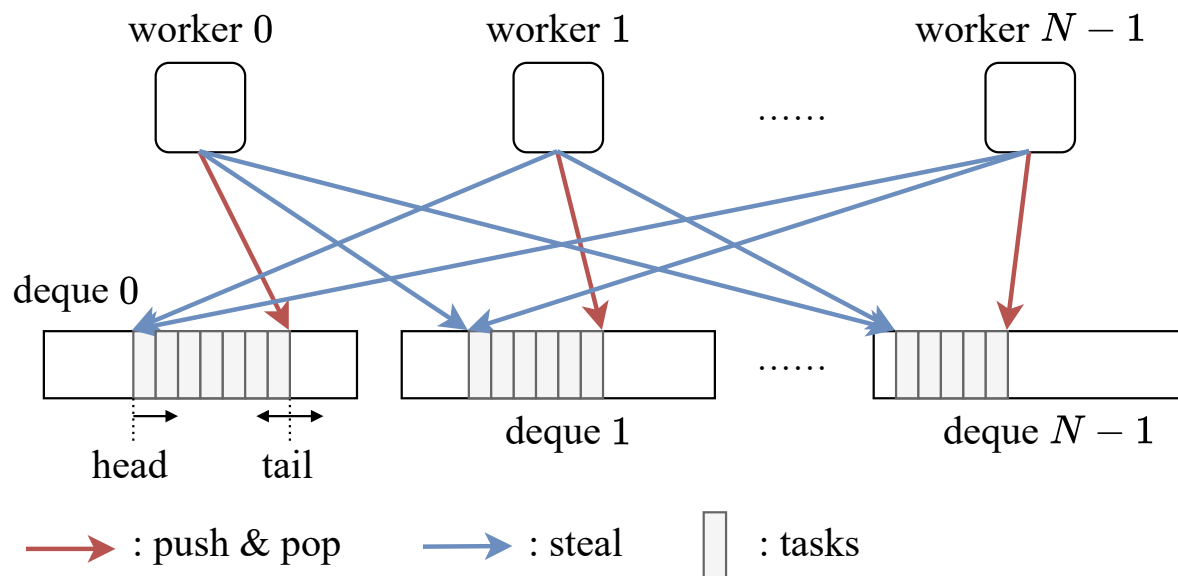
[補足] タスク並列のスケジューリング (1/3)

- タスク並列では、不規則性ゆえに**静的負荷分散**ではなく**動的負荷分散**が通常用いられる
- 一番シンプルなのが、共有キュー方式
 - 共有のタスクキュー（実行可能なタスクを格納するキュー）が一個存在し、全ワーカーがそこにタスクを push/pop する
 - キューアクセス時の競合が大きい



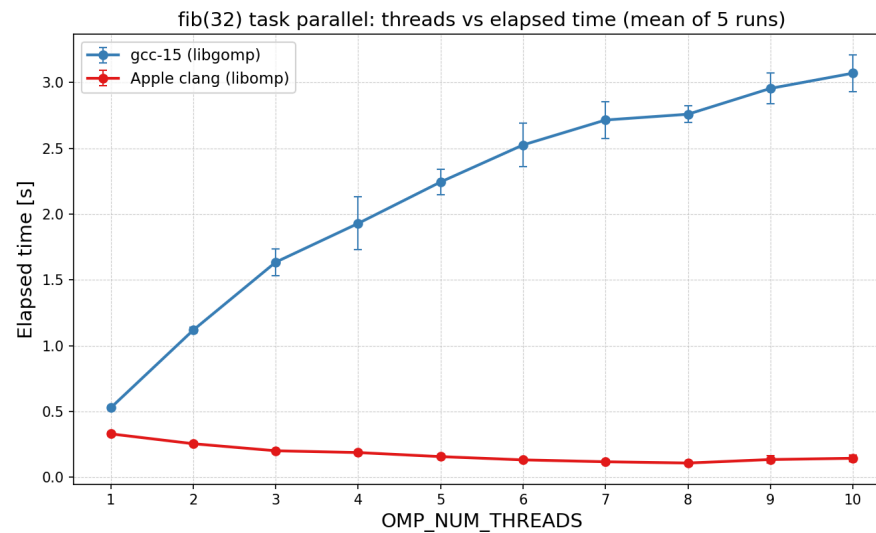
[補足] タスク並列のスケジューリング (2/3)

- 一方で、それよりも高速な動的負荷分散方式として **work stealing** がある
- 各ワーカーは 1 つずつタスクキューを保持する
 - 自身のキューが空になった場合、他のキューからタスクを steal
 - キューアクセス時の競合を緩和できるため、**ワーカー数の増加に対してより良好にスケール**することが知られている



[補足] タスク並列のスケジューリング (3/3)

- OpenMP の仕様上, 具体的なスケジューリング方針は定められておらず, 実装によって異なる
- [要検証] ChatGPT 曰く
 - GCC libgomp : 共有キュー方式
 - LLVM libomp : work stealing
- 実際に fib で OMP_NUM_THREADS をスケールさせながら測定してみる

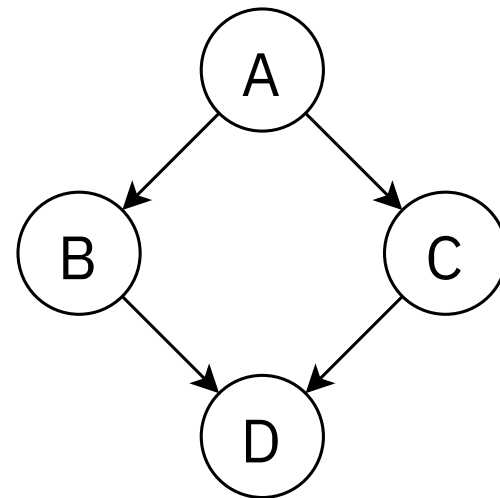


- LLVM libomp は良好にスケールする一方で, GCC libgomp はスレッドを増やすほど遅くなる

[補足] task depend

- depend 節により、タスク間の依存関係を指定できる
- 依存関係を持つ処理を DAG として表現できる
 - 依存関係が満たされたタスクから順に実行される
 - 単なる fork-join による同期よりも細粒度の制御が可能になる

```
#pragma omp task depend(out: A)  
compute_A();  
#pragma omp task depend(in: A) depend(out: B)  
compute_B();  
#pragma omp task depend(in: A) depend(out: C)  
compute_C();  
#pragma omp task depend(in: B, C)  
compute_D();
```



- OpenMP の task は、共有メモリ環境におけるタスク並列
- 複数ノードにまたがる分散メモリ環境でタスク並列を実現するのは簡単ではない
 - タスクをどのノードで実行するか
 - 必要なデータをどこに配置するか
 - ノード間通信をどのタイミングで行うか
 - 負荷分散をどのように行うか
- Itoyori は、このような**分散メモリ環境でのタスク並列を扱う処理系**の一例 ([Paper](#), [GitHub](#))

[補足] GPU タスク並列処理系 GTaP

- GTaP は, GPU 上でタスク並列を扱う処理系 ([Paper](#), [GitHub](#))
- GPU 上に常駐する runtime が, タスクの生成・実行・同期を管理する
 - CPU から細かい GPU カーネルを何度も起動するのではなく, GPU 内でタスクをスケジューリング
 - `#pragma gtap task` / `#pragma gtap taskwait` により, OpenMP task に近い形で記述できる
 - GPU で fork-join をサポートするのはそんなに簡単ではない
- 2つの実行モードをサポートする
 - **1スレッド1タスク**: 細粒度で不規則なタスクを, 多数の GPU スレッドで実行する
 - **1スレッドブロック1タスク**: タスク内部にデータ並列性がある場合に, 1つのブロック内のスレッドが協調して実行する

目次

OpenMP とは？

ループ並列化

正しく並列化するために

速く並列化するために

その他の構文

▶ **Appendix**

▶ **SIMD**

OpenMP の GPU 対応

プログラム例

各種ディレクティブの実装

- SIMD を用いると、同じ命令を複数のデータに対して並列に実行できる
- OpenMP では `#pragma omp simd` で SIMD 機能を使用できる

```
#pragma omp simd  
for (int i = 0; i < N; i++) {  
    b[i] = 2 * a[i];  
}
```

参考：コンパイルと最適化レポート

- 以下のようにコンパイルすることで、ベクトル化に関する最適化レポートを `vec_report.txt` に出力できる
- `-O3` は最適化レベルを指す

```
$ gcc-15 -O3 -fopt-info-vec-optimized=vec_report.txt -fopenmp simd.c -o simd
```

- 出力例：

```
simd.c:30:21: optimized: loop vectorized using 16 byte vectors  
simd.c:30:21: optimized: loop vectorized using 8 byte vectors  
simd.c:18:10: optimized: loop vectorized using 16 byte vectors
```

- たしかに SIMD 化されていることがわかる

参考：アセンブリの生成

- 以下のようにコンパイルすることで、アセンブリを `simd.s` に出力できる

```
$ gcc-15 -O1 -fopenmp -S simd.c -o simd.s
```

- 出力例：

```
ldr q31, [x7, x1]           // x7 + x1 のアドレスから128bitロード (int型4つ分)
add v31.4s, v31.4s, v31.4s  // v31の4つの32bit整数をそれぞれ2倍 (ベクトル加算)
str q31, [x0, x1]           // x0 + x1 のアドレスに128bitストア (結果を書き込み)
```

- ベクトルレジスタにデータを格納後、4つの整数を一気に2倍している
- Mac の場合、Arm Neon という SIMD 拡張が採用されているようである
- SIMD 命令のために用意されているレジスタ幅が 128bit のため、32bit 整数 4 つや 64bit 小数 2 つを一気に演算できる

SIMD の有無による実行時間の比較

- make bench-simd で各条件 10 回実行し, 平均 $\pm$ 標準偏差を表示
 - 最適化レベル -O1 のうえに #pragma omp simd を指定/非指定
 - そのうえで, アセンブリで SIMD 命令が有効/無効化されていることを確認
- SIMD ありの場合

```
$ make bench-simd
# SIMD (simd), 10 runs each
OMP_NUM_THREADS=1: 0.032581  $\pm$  0.000935
OMP_NUM_THREADS=4: 0.015092  $\pm$  0.000373
```

- SIMD なしの場合

```
# scalar (simd_scalar), 10 runs each
OMP_NUM_THREADS=1: 0.048844  $\pm$  0.001819
OMP_NUM_THREADS=4: 0.017411  $\pm$  0.000664
```

SIMD の有無による実行時間の比較

- 上記は各条件 10 回測定の平均；標準偏差も併記している
- ただし，SIMD 化しても単純にベクトル幅の分だけ速くなるとは限らない
 - メモリアクセス，キャッシュ，ループ制御，命令スケジューリングなどの影響を受ける
 - 特にスレッド数を増やすと，メモリ帯域など他の要因がボトルネックになることがある
- なお，自分の環境・このプログラムでは，そもそも最適化レベルを -O3 にしてコンパイルすると，`#pragma omp simd` を指定しなくても SIMD 命令が有効化された

目次

OpenMP とは？

ループ並列化

正しく並列化するために

速く並列化するために

その他の構文

▶ **Appendix**

SIMD

▶ **OpenMP の GPU 対応**

プログラム例

各種ディレクティブの実装

- OpenMP 4.0 以降, GPU などのアクセラレータ向けの機能が追加された
- target 指示文を使うことで GPU での実行が可能
- 似たようなアクセラレータ向けの指示文 API として **OpenACC** がある
 - ACC は Accelerator を意味する
 - OpenMP と異なり, アクセラレータ向けに最適化された設計

OpenMP GPU offloading の基本モデル

```
#pragma omp target teams distribute parallel for \  
           map(to: A[0:N], B[0:N]) map(from: C[0:N])  
for (int i = 0; i < N; i++) {  
    C[i] = A[i] + B[i];  
}
```

- target : GPU 側で実行する
- teams : GPU 上に複数の team を作る
- distribute : ループ反復を team 間に分配する
- parallel for : team 内のスレッドでループを並列実行する
- map : host と device 間でデータを転送する

GPU 利用時の注意：データ転送コスト

- GPU で計算しても、毎回データ転送を行うと性能が出ないことがある
 - host メモリから device メモリへ入力データを転送
 - device 上で計算
 - device メモリから host メモリへ結果を転送
- 小さい計算を何度も GPU に投げると、計算時間より転送・起動オーバーヘッドが支配的になりやすい

```
#pragma omp target data map(to: A[0:N], B[0:N]) map(from: C[0:N])
{
    #pragma omp target teams distribute parallel for
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }

    // 必要なら、この中で複数回GPU計算を行う
}
```

目次

OpenMP とは？

ループ並列化

正しく並列化するために

速く並列化するために

その他の構文

▶ **Appendix**

SIMD

OpenMP の GPU 対応

▶ **プログラム例**

各種ディレクティブの実装

for 並列のプログラム例：行列積

- 行列積を OpenMP の for 並列で実装した例（チューニング済み）
 - ループ入れ替え (i-k-j) で A の行・B の行を順にアクセス
 - ブロック化 (TILE) で L1/L2 キャッシュを有効利用
 - 外側ループを `#pragma omp parallel for` で並列化
- `matmul_tuned.c` を参照

```
#pragma omp parallel for schedule(static)
for (int ii = 0; ii < N; ii += TILE) {
    for (int kk = 0; kk < N; kk += TILE) {
        for (int jj = 0; jj < N; jj += TILE) {
            for (int i = ii; i < i_end; i++) {
                for (int k = kk; k < k_end; k++) {
                    double a_ik = A[i][k];
                    for (int j = jj; j < j_end; j++) {
                        C[i][j] += a_ik * B[k][j];
                    }
                }
            }
        }
    }
}
```


task 並列のプログラム例：merge sort, cilk sort

- 再帰的なソートは，OpenMP の task と相性がよい
 - 配列を分割し，部分配列のソートをタスクとして生成する
 - 小さくなった部分配列は，タスク化せず逐次ソートする
 - 例：挿入ソート，逐次 merge sort など
 - 子タスクの完了を `taskwait` で待ってから，結果をマージする
- 単純な並列 merge sort：
 - 左半分と右半分のソートをそれぞれタスク化
 - 最後のマージは逐次的に行う
- Cilk sort：
 - **ソートだけでなく，マージ処理も再帰的に分割してタスク化する**
 - より細かく並列性を取り出せる
- `parallel_merge_sort.c`, `cilk_sort.c` を参照

GPU 実行のプログラム例：行列積

- 行列積の GPU での並列化を OpenMP で実装する
 - 最適化はなし，シンプルなもの
- `matmul_gpu.c` を参照

```
#pragma omp target data map(to:A[0:N][0:N], B[0:N][0:N]) map(from:C[0:N][0:N])
{
    #pragma omp target teams distribute parallel for collapse(2)
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            for(int k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

GPU 実行のプログラム例：行列積

- ここでは collapse(2) の有無による実行時間を比較してみる（miyabi-g 上で実行）
- 実行結果の例：

```
$ ./matmul_gpu
Use collapse
Test passed!
Computation time: 0.386624 seconds
```

```
$ ./matmul_gpu
No collapse
Test passed!
Computation time: 0.702773 seconds
```

- collapse によりイテレーション個数が N から N^2 が増え、GPU 上の多数のスレッドに計算が行き渡ると考えられる

目次

OpenMP とは？

ループ並列化

正しく並列化するために

速く並列化するために

その他の構文

▶ **Appendix**

SIMD

OpenMP の GPU 対応

プログラム例

▶ **各種ディレクティブの実装**

- 誤りを含む可能性があります
- 以下 gcc における OpenMP ランタイム `libgomp` の内部実装に限って話を進める
- 実装は <https://github.com/gcc-mirror/gcc/tree/releases/gcc-9.2.0/libgomp> を参照

各種ディレクティブの実装①：スレッドの生成

- parallel ディレクティブは以下のように展開される

```
void subfunction (void *data)
{
    use data;
    body;
}

setup data;
GOMP_parallel_start (subfunction, &data, num_threads);
subfunction (&data);
GOMP_parallel_end ();
```

- GOMP_parallel_start が gomp_team_start() を呼び出し、OS レベルのスレッドを生成 (Linux, Mac : pthread_create)
- git リポジトリの parallel.c, team.c 参照

各種ディレクティブの実装②：lockの実装

- 自分の調べた範囲では pthread_mutex と同等
- config/posix/mutex.h に以下のように定義されている

```
typedef pthread_mutex_t gomp_mutex_t;  
static inline void gomp_mutex_init (gomp_mutex_t *mutex)  
{  
    pthread_mutex_init (mutex, NULL);  
}  
static inline void gomp_mutex_lock (gomp_mutex_t *mutex)  
{  
    pthread_mutex_lock (mutex);  
}  
// 略
```

- lock.c 参照

各種ディレクティブの実装③：criticalの実装

- 名前なし critical セクションの定義

```
void
GOMP_critical_start (void)
{
    /* There is an implicit flush on entry to a critical region. */
    __atomic_thread_fence (MEMMODEL_RELEASE);
    gomp_mutex_lock (&default_lock);
}

void
GOMP_critical_end (void)
{
    gomp_mutex_unlock (&default_lock);
}
```

- 結局 mutex で管理していそう
- critical.c 参照

各種ディレクティブの実装④：atomicの実装

- atomic1.c をアセンブリにすると, atomic 部分は以下のようなになる

```
ldadd w1, w0, [x0]
```

- arm において ldadd は「メモリ位置 [x0] から値をロード→w0 の値を加算→結果を [x0] にストア→元のメモリ値を w1 に返す」を**不可分**で実行する命令

各種ディレクティブの実装④：atomicの実装

- もう少し複雑だとどうなるか
- atomic2.c をアセンブリにすると, atomic 部分は以下ようになる

```
LFB0:
    fadd    d0, d0, d0        // bを2倍
    ldr     x1, [x0]          // *aをx1に読み出し
L2:
    fmov    d31, x1           // x1を浮動小数点レジスタd31に移動
    mov     x2, x1            // x1をx2にコピー
    fadd    d31, d0, d31      // d31 = *a + 2*b
    fmov    x3, d31           // x3に新しい値を格納
    cas     x2, x3, [x0]      // メモリ内の現値[x0]がx2（旧値）と一致すればx3（新値）をatomicに書き込み
                                // いずれの場合も元のメモリ値をx2に戻す
    cmp     x1, x2            // casの結果, x1==x2なら成功
    bne     L3                // 失敗なら再試行
    ret
```

- CAS ループになっている